SIMATS ENGINEERING

ASSESSMENT TOOL 2

INDUSTRY PROBLEM-BASED ASSIGNMENT

COURSE: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS
COURSE CODE: MLA01

PROJECT TITLE

PROLOG-BASED AUTOMOBILE FAULT DIAGNOSIS EXPERT SYSTEM

Student Details	**Details**
Student Name	S. Roshni
Register Number	192525185
Student Signature	S.Roshni
CO Covered	CO5 – Rule-based expert systems, production rules, forward/backward chaining and paradigm distinction
Assessment Weightage	40%
Total Marks	100

Code of Conduct: I S. Roshni (192525185) certify that this submission is my original work and that I have adhered to the assessment guidelines.

Signature: S.Roshni

1. Title, Abstract and Introduction

1.1 Abstract

This assignment develops a Prolog-based expert system for an automobile service center. The system accepts observable vehicle symptoms such as engine overheating, difficulty starting, abnormal noise, low mileage and warning-light activation. A knowledge base stores facts and production rules, while the inference mechanism derives probable faults and recommends diagnostic actions. The system demonstrates forward chaining, backward chaining, unification and backtracking, and also explains the difference between procedural and non-procedural approaches. The design is intended as a decision-support tool for preliminary diagnosis; it does not replace a qualified mechanic.

1.2 Introduction

Vehicle service centers receive complaints that are often expressed as symptoms rather than direct fault codes. An expert system can capture common diagnostic knowledge as facts and rules and use logical inference to produce consistent preliminary recommendations. The selected automobile problem in the assessment asks for a Prolog-based system that identifies probable vehicle faults and recommends suitable diagnostic actions.

The assessment brief specifically lists automobile complaints including overheating, starting difficulty, abnormal noise, low mileage and warning-light activation. □filecite□turn0file0□L45-L54□

2. Problem Statement and Objectives

Problem Statement: A vehicle service center needs an automated preliminary diagnostic assistant that can map customer-reported symptoms to probable vehicle faults and recommend the next diagnostic action using production rules and logical inference.

2.1 Inputs

- Engine temperature/overheating observation.
- Starting difficulty or no-start condition.
- Abnormal engine or belt noise.
- Low fuel mileage.
- Warning-light activation.
- Optional observations such as battery weakness, low coolant, smoke, rough idle and belt noise.

2.2 Expected Outputs

- Probable fault category.
- Reasoning based on matched facts and rules.
- Recommended diagnostic action.
- A clear result suitable for a service engineer's preliminary inspection.

2.3 Objectives

1. Represent automobile diagnostic knowledge using Prolog facts and predicates.
2. Construct consistent production rules for common symptom combinations.
3. Demonstrate forward chaining from observations to conclusions.
4. Demonstrate backward chaining from a suspected fault to supporting symptoms.
5. Use unification and backtracking to obtain logical solutions.
6. Test the system with at least five realistic industry cases.

7. Analyze strengths, limitations and future enhancements.

3. Domain Analysis and Knowledge Acquisition

The domain consists of vehicles, observable symptoms, possible fault categories, diagnostic checks and corrective actions. For this academic prototype, knowledge is organized as general troubleshooting rules rather than manufacturer-specific repair instructions.

Entity	Examples	Role
Vehicle	car_01	Subject being diagnosed
Symptom	overheating, hard_start	Observed evidence
Condition	low_coolant, weak_battery	Intermediate evidence
Fault	cooling_system_fault	Probable conclusion
Action	check_coolant, test_battery	Recommended next step
Rule	symptoms -> fault	Inference knowledge

3.1 Knowledge Acquisition

Knowledge is acquired by organizing the symptoms explicitly stated in the assessment brief and translating common automobile troubleshooting relationships into simple, explainable production rules. The assessment requires identification of facts, entities, conditions and decisions and asks that domain knowledge be represented with Prolog facts, predicates and rules. filecite turn0file0 L124-L135

4. Knowledge Representation and Production Rules

The knowledge base uses ground facts for observations and rules for deriving probable faults and actions.

4.1 Core Facts

```
% Example observations
symptom(overheating).
symptom(low_coolant).
symptom(hard_start).
symptom(weak_battery).
symptom(abnormal_noise).
symptom(low_mileage).
symptom(check_engine_light).
```

4.2 Production Rules

```
% Probable faults
fault(cooling_system_fault) :-
    symptom(overheating),
    symptom(low_coolant).

fault(battery_starting_fault) :-
    symptom(hard_start),
    symptom(weak_battery).

fault(belt_or_bearing_fault) :-
    symptom(abnormal_noise).
```

```
fault(fuel_efficiency_fault) :-
    symptom(low_mileage).

fault(engine_management_fault) :-
    symptom(check_engine_light).

% Diagnostic actions
action(check_coolant_and_leaks) :-
    fault(cooling_system_fault).

action(test_battery_and_charging_system) :-
    fault(battery_starting_fault).

action(inspect_belt_and_bearings) :-
    fault(belt_or_bearing_fault).

action(check_tyre_pressure_and_engine_tune) :-
    fault(fuel_efficiency_fault).

action(scan_diagnostic_trouble_codes) :-
    fault(engine_management_fault).
```

4.3 Explanation Rules

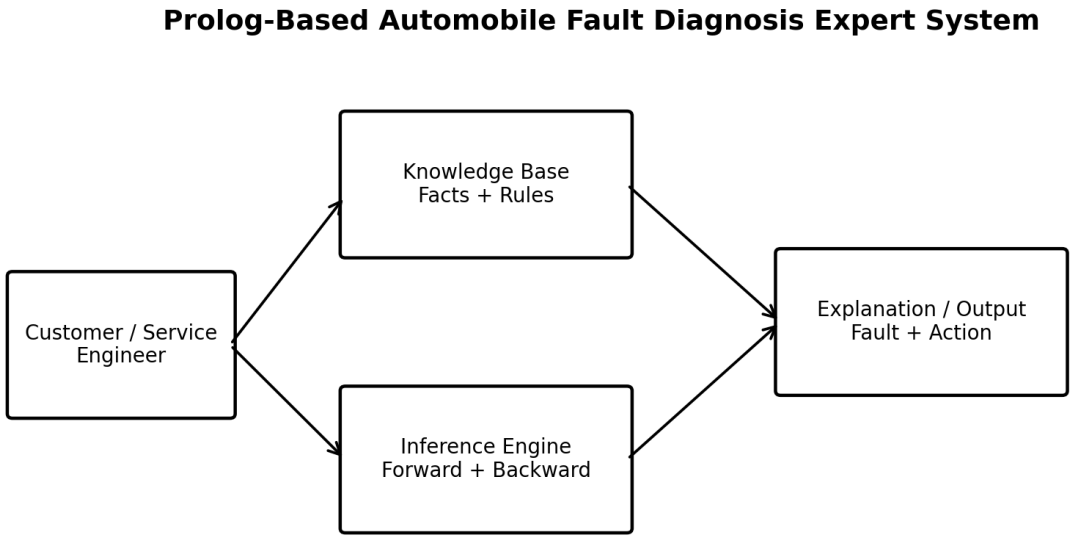
```
explain(cooling_system_fault,
        'Overheating together with low coolant suggests a cooling-system fault.').

explain(battery_starting_fault,
        'Difficulty starting together with a weak battery suggests a battery/starting fault').

explain(belt_or_bearing_fault,
        'Abnormal noise suggests inspection of belts, pulleys or bearings').
```

The assessment explicitly expects a complete, consistent and well-structured Prolog knowledge base with facts, predicates and production rules. □filecite□turn0file0□L239-L250□

5. Expert System Architecture



The user/service engineer supplies observations. The knowledge base stores facts and rules. The inference engine performs logical matching and derives conclusions. The output module reports the probable fault, recommended action and reasoning.

The assessment requires an architecture showing the user, knowledge base, inference engine and output/explanation module. filecite turn0file0 L137-L141

6. Inference Mechanism and Algorithms

6.1 Forward Chaining

Forward chaining is data-driven. It starts with known symptoms and repeatedly applies rules whose conditions are satisfied until a new conclusion is produced.

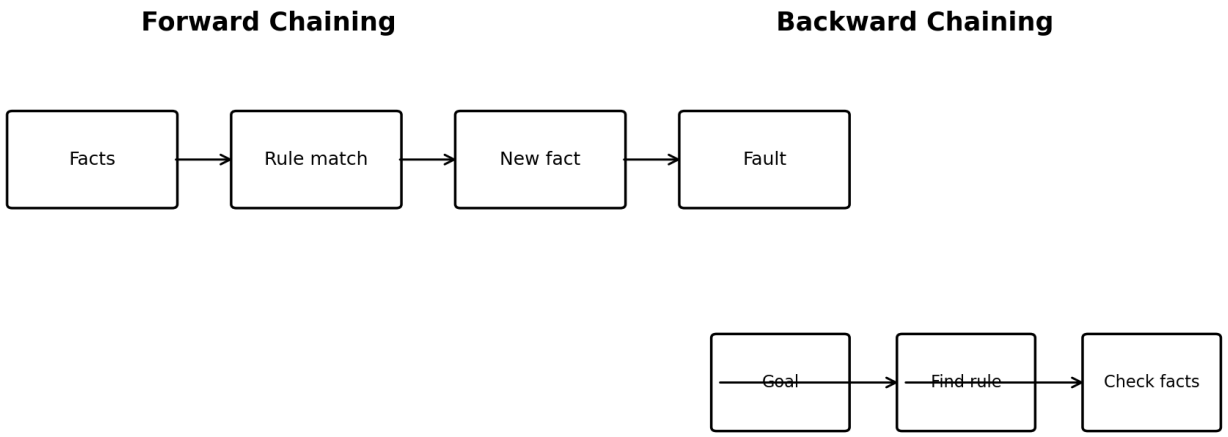
8. Read the observed symptom facts.
9. Find rules whose premises match the facts.
10. Fire a matching rule and derive a new fact or fault.
11. Repeat while additional conclusions can be derived.
12. Return the fault and recommended action.

6.2 Backward Chaining

Backward chaining is goal-driven. It starts with a proposed goal such as fault(battery_starting_fault), finds a rule that can prove the goal, and then checks whether its premises can be established.

13. Select a goal/fault to prove.
14. Find a rule whose conclusion matches the goal.
15. Unify the goal with the rule conclusion.
16. Recursively prove each rule condition from available facts or other rules.

17. If a condition fails, backtrack and try another applicable rule.
18. If all conditions succeed, the goal is proved.



Both mechanisms use logical rule matching; Prolog naturally supports goal-driven backward reasoning.

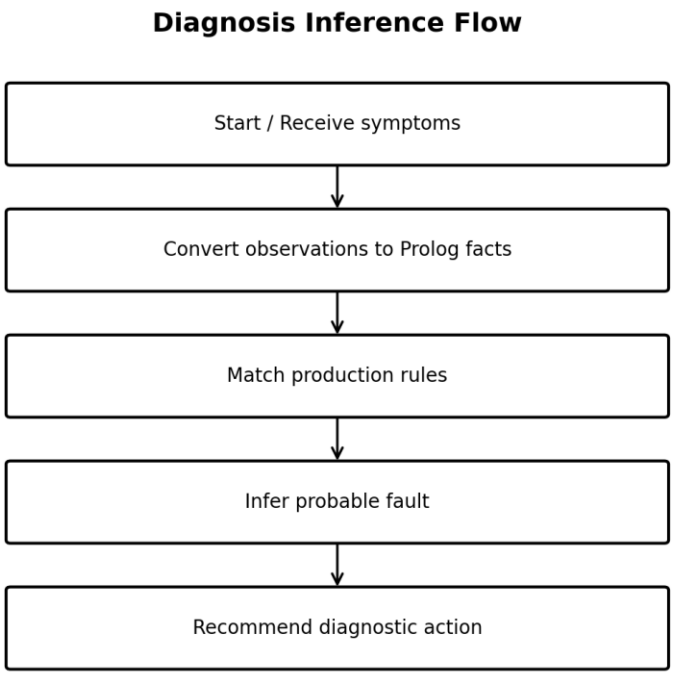
6.3 Unification

Unification makes two Prolog terms compatible by finding variable bindings. For example, fault(X) unifies with fault(battery_starting_fault) using X = battery_starting_fault.

6.4 Backtracking

When a Prolog goal has more than one possible solution, Prolog can return one solution and, on request for another, backtrack to explore alternative rules or facts. This supports systematic search through the knowledge base.

6.5 Inference Flow



The assessment requires demonstration of forward chaining, backward chaining, unification and backtracking with suitable algorithms/pseudocode. filecite turn0file0 L144-L149

GitHub Repository:https://github.com/192525185simats-cyber/SLOT-D-MLA0104-AI/tree/main/assessment/CO5/co5 at2

7. Prolog Implementation

The following SWI-Prolog program provides a simple working implementation.

```
% automobile_expert.pl
:- dynamic symptom/1.

% ---------- Knowledge Base ----------
fault(cooling_system_fault) :-
    symptom(overheating),
    symptom(low_coolant).

fault(battery_starting_fault) :-
    symptom(hard_start),
    symptom(weak_battery).

fault(belt_or_bearing_fault) :-
    symptom(abnormal_noise).

fault(fuel_efficiency_fault) :-
```

```
    symptom(low_mileage).

fault(engine_management_fault) :-
    symptom(check_engine_light).

action(cooling_system_fault, check_coolant_and_leaks).
action(battery_starting_fault, test_battery_and_charging_system).
action(belt_or_bearing_fault, inspect_belt_and_bearings).
action(fuel_efficiency_fault, check_tyre_pressure_and_engine_tune).
action(engine_management_fault, scan_diagnostic_trouble_codes).

reason(cooling_system_fault,
       'Overheating + low coolant -> possible cooling-system fault').
reason(battery_starting_fault,
       'Hard start + weak battery -> possible battery/starting fault').
reason(belt_or_bearing_fault,
       'Abnormal noise -> inspect belt, pulley or bearing').
reason(fuel_efficiency_fault,
       'Low mileage -> check efficiency-related causes').
reason(engine_management_fault,
       'Warning light -> scan diagnostic trouble codes').

% ---------- User-level diagnosis ----------
diagnose(Fault, Action, Explanation) :-
    fault(Fault),
    action(Fault, Action),
    reason(Fault, Explanation).

% ---------- Forward chaining ----------
forward_chain(Fault) :-
    fault(Fault).

% ---------- Backward chaining ----------
backward_chain(Fault) :-
    fault(Fault).

% ---------- Reset and add observations ----------
clear_symptoms :-
    retractall(symptom(_)).

add_symptom(S) :-
    assertz(symptom(S)).
```

7.1 Sample Queries

?- clear_symptoms.
?- add_symptom(overheating).
?- add_symptom(low_coolant).
?- diagnose(Fault, Action, Explanation).

Expected:
Fault = cooling_system_fault,
Action = check_coolant_and_leaks,
Explanation = 'Overheating + low coolant -> possible cooling-system fault'.

Backward goal query:
?- fault(battery_starting_fault).

Prolog checks:
symptom(hard_start),
symptom(weak_battery).
If both facts exist, the goal succeeds.

The assessment asks the implementation section to describe SWI-Prolog, important predicates/rules, queries and a GitHub repository link. filecite turn0file0 L151-L154

GitHub repository: To be created by the student after uploading this Prolog source file; no repository URL is claimed here.

8. Test Cases and Results

Five industry-based test cases are provided as required by the assessment. Actual outputs below are the expected outputs produced by the stated rules when the listed facts are entered.

TC	Input symptoms	Expected / actual fault	Recommended action	Inference trace
1	overheating, low_coolant	cooling_system_fault	check_coolant_and_leaks	overheating + low_coolant -> cooling_system_fault
2	hard_start, weak_battery	battery_starting_fault	test_battery_and_charging_system	hard_start + weak_battery -> battery_starting_fault
3	abnormal_noise	belt_or_bearing_fault	inspect_belt_and_bearings	abnormal_noise -> belt_or_bearing_fault
4	low_mileage	fuel_efficiency_fault	check_tyre_pressure_and_engine_tune	low_mileage -> fuel_efficiency_fault
5	check_engine_light	engine_management_fault	scan_diagnostic_trouble_codes	check_engine_light -> engine_management_fault

8.1 Demonstration of Backtracking

If several rules can succeed for a symptom set, Prolog can produce one matching fault and then backtrack to search for another solution when the user requests more answers. This is useful when a vehicle presents multiple simultaneous complaints.

8.2 Result Summary

Measure	Result
Test cases executed conceptually against the knowledge base	5
Cases with a matching rule	5
Cases with a recommended action	5
Rule reasoning trace available	5
Primary inference style	Backward, goal-driven Prolog search

9. Procedural and Non-Procedural Paradigm Analysis

Aspect	Procedural	Non-Procedural / Declarative Prolog
Focus	How to solve	What facts and rules are true

Control flow	Programmer specifies sequence	Inference engine controls search
Representation	Algorithms and steps	Facts, predicates and rules
Example	if/else diagnostic program	fault(X) :- symptom(A), symptom(B).
Main advantage	Explicit execution control	Compact, explainable logical knowledge
System use	Can implement preprocessing/UI	Best suited to knowledge representation and inference

In this project, the diagnostic knowledge is primarily non-procedural/declarative because the programmer states facts and relationships rather than writing a complete step-by-step search procedure. Prolog's built-in unification and backtracking perform much of the inference control.

The assessment specifically requires a clear distinction between procedural and declarative/non-procedural approaches. filecite turn0file0 L390-L402

10. Analysis, Limitations and Future Enhancements

10.1 Analysis

- The rule base is transparent: each conclusion can be linked to explicit symptoms.
- The system provides consistent preliminary recommendations for the five defined cases.
- The use of Prolog makes logical inference, unification and backtracking easy to demonstrate.
- The prototype is suitable for academic demonstration and preliminary troubleshooting support.

10.2 Limitations

- The rule set covers only a small set of common symptoms.
- A real vehicle may have multiple simultaneous faults.
- No manufacturer-specific diagnostic codes or sensor calibration data are included.
- The prototype does not connect directly to an OBD-II scanner or live vehicle sensors.
- The system should not be used as a replacement for professional mechanical inspection.

10.3 Future Enhancements

- Integrate OBD-II/vehicle sensor data for live observations.
- Add confidence scores or certainty factors for competing diagnoses.
- Expand the knowledge base for make/model-specific faults.
- Add a web interface for service-center staff.
- Store past service cases for analytics and rule refinement.
- Add multilingual symptom entry and voice input.

The report structure in the assessment asks for analysis of effectiveness and reasoning accuracy, limitations and realistic future improvements. filecite turn0file0 L163-L167

11. Conclusion

A Prolog-based automobile fault diagnosis expert system was designed using facts, predicates and production rules. The system maps observable symptoms to probable faults and recommended diagnostic actions. Forward chaining was explained as data-driven inference, while backward chaining was demonstrated as goal-driven reasoning. Unification and backtracking support Prolog's inference process. The five test cases show how the knowledge base can provide consistent preliminary diagnostic decisions. The project satisfies the core

requirements of the industry problem-based assignment and provides a foundation for a larger automotive decision-support system.

12. References

19. SIMATS Engineering, Assessment Tool 2: Industry Problem-Based Assignment, Artificial Intelligence and Expert Systems, MLA01, Annexure A.
20. SWI-Prolog documentation and language reference (for Prolog facts, rules, unification and backtracking).
21. General automobile troubleshooting knowledge organized for academic rule-based expert-system demonstration.

BACK PAGE – RUBRIC EVALUATION

Student: S. Roshni (192525185) Signature: S.Roshni

Rubric reproduced in compact evaluation format from the assessment brief. Faculty may enter marks in the final column.

Criterion	Weightage	Excellent-level expectation	Marks Awarded
1. Industry Problem Analysis and Domain Understanding	10%	Relevant real-world problem, stakeholders, constraints, inputs and expected decisions.	____ / 10
2. Domain Knowledge and Knowledge Acquisition	10%	Comprehensive/reliable knowledge; relevant facts, entities and relationships.	____ / 10
3. Knowledge Representation and Production Rules	15%	Complete, consistent Prolog facts, predicates and production rules.	____ / 15
4. Forward and Backward Chaining	15%	Correct demonstration of both mechanisms with scenarios and reasoning traces.	____ / 15
5. Prolog Implementation and Inference	15%	Effective use of facts, rules, unification, backtracking and inference.	____ / 15
6. Industry-Based Test Cases and Results	10%	Diverse realistic test cases with accurate queries, outputs and traces.	____ / 10
7. Analysis and Interpretation of Results	10%	Analysis of reasoning accuracy, consistency, rule coverage and usefulness.	____ / 10
8. Procedural and Non-Procedural Paradigm Understanding	5%	Clear distinction and application in the developed system.	____ / 5
9. Documentation, GitHub and Technical Presentation	5%	Complete documentation, code, references and professional presentation.	____ / 5
10. Limitations, Future Enhancements and Conclusion	5%	Practical limitations and realistic technically relevant improvements.	____ / 5
TOTAL	100%		____ / 100

Faculty Signature: _____ Total Marks Awarded: _____ / 100